

Refactoring Report: Homework 3 - Maximal Update Parameterization

Vongani Maluleke and Gemini AI

December 4, 2025

1 Introduction

This report details the refactoring changes applied to the provided Python code, which already included the completed logic for Maximal Update Parameterization (muP), per-weight multipliers, and Simplified Shampoo. The goal was to enhance the code's readability, type safety, and Pythonic style while strictly preserving the existing functionality and flow required for the educational context. Gemini was used to assist with the refactoring by prompting it to write clean and pythonic code. The code can be found here: Homework3

2 Refactoring Changes and Citations

2.1 General Code Style and Type Safety

- **Type Hinting:** Explicit type hints were added to all function and method signatures (`List[int]`, `torch.Tensor`, `-> None`, `-> float`, etc.). This aligns with modern Python best practices (**PEP 484**) [1].
- **Modern Imports:** Imports were consolidated from `import torch.nn as nn` and `import torch.optim as optim` to `from torch import nn, optim`.
- **Boolean Checks:** Used `p.ndim` instead of `len(p.shape)` when checking if a parameter is a weight matrix or a bias vector.

2.2 MLP Class Refactoring

- **List Comprehension:** The iterative population of `self.layers` in `MLP.__init__` was replaced with a concise list comprehension.
- **Tensor Flattening:** `x.view(x.size(0), -1)` was replaced by the more semantically clear `x.flatten(start_dim=1)` in `MLP.forward`.

2.3 SimpleAdam and SimpleAdamMuP Optimizers

The core logic for Adam remains, but the structure was made cleaner.

- **The Walrus Operator (`:=`):** Used for concise initialization of the state dictionary within the `step` method. The expression `if (step_tensor := state.get("step")) is None` reduces code duplication and improves flow (**PEP 572**) [2].
- **Parameter Unpacking:** Group parameters (`lr`, `b1`, `b2`, `epsilon`) are unpacked at the start of the outer loop for cleaner access within the inner parameter loop, replacing repeated calls to `group["lr"]`.
- **Epsilon Standardization:** Added `epsilon` to the `defaults` dictionary in `SimpleAdam` and passed it to the denominator (`v_hat.sqrt() + epsilon`) for consistency and to remove the hardcoded `1e-16`.

2.4 MuP Scaling Logic (SimpleAdamMuP)

The logic for calculating `mu_lr` was made more explicit and robust by using a `param_index` counter to determine the layer type (input, hidden, output) based on the sequential ordering of parameters, rather than solely relying on shape heuristics.

- **Layer Indexing:** A `param_index` counter was introduced to correctly identify parameters, which is more reliable than shape checks alone (e.g., distinguishing between hidden layer biases and the output bias).
- **Clarity in Scaling:** The temporary `mu_lr` variable is used to hold the layer-specific learning rate before being applied to `p.add_` for clarity.

2.5 Simplified Shampoo Optimizers

- **SVD Notation:** Standard linear algebra notation $\mathbf{U}, \mathbf{S}, \mathbf{V}^T$ was used in the SVD call and assignment (`U, S, VT = torch.linalg.svd(m, full_matrices=False)`) for better academic and domain-specific clarity.
- **Dimension Check:** Used `m.ndim == 1` for checking if the momentum `m` belongs to a bias vector, which is more Pythonic than checking the length of the shape tuple.

3 Summary of Key Pythonic Refactoring

Original Code	Refactored Code	Style Improvement
<pre>if len(state) == 0: for i in range(len(L)): L.append(...) x.view(x.size(0), -1) def func(a, b): torch.sqrt(v_hat) + 1e-16</pre>	<pre>if (step := state.get("step")) is None: [... for i in range(len(L))] x.flatten(start_dim=1) def func(a: int, b: str) -> float: v_hat.sqrt() + epsilon</pre>	PEP 572 (Walrus Operator) [2] List Comprehension Semantic Clarity (PyTorch) Explicit Type Hinting (PEP 484) [1] Using Unpacked Variable (<code>epsilon</code>)

Table 1: Examples of Pythonic Refactoring

References

- [1] Van Rossum, G., Lehtosalo, J., & Langa, L. (2015). **PEP 484 – Type Hints.** *Python Enhancement Proposal*. <https://peps.python.org/pep-0484/>
- [2] Angelico, C., Peters, T., & van Rossum, G. (2018). **PEP 572 – Assignment Expressions.** *Python Enhancement Proposal*. <https://peps.python.org/pep-0572/>